

ANDROID APPLICATION DEVELOPMENT

Webskitters Technology Solutions Private Ltd.

Session: 2018 –2019

A Industrial Training Report Submitted To
SIKKIM MANIPAL INSTITUTE OF TECHNOLOGY,
MAJITAR, SIKKIM

Towards Partial Fulfillment of Degree of

Bachelor of Technology

In

ELECTRONICS AND COMMUNICATION Engineering

Submitted To:

Submitted By:

Candidate Name: HARSH AGARWAL

Registration No: 201600622

Department of Electronics and Communication Engineering

CERTIFICATE



This is to certify that the Industrial Training report entitled **“AndroidApplication Development”** submitted by **---Harsh Agarwal, 201600622, B.Tech** in the year 2018-19 of **Electronics and Communication Engineering** department of this institute is a satisfactory account of his Industrial Training work based on syllabus which is approved for the award of degree of Bachelor of Technology.

Internal Examiner

Acknowledgement

I express my sincere gratitude towards my guide Mr Swarup Kumar Saha (Senior Technical Analyst) of Webskitters Technology Solutions Private Ltd. for his constant motivation throughout the course of this training.

I would also like to thank the trainers in the Webskitters Academic Institution, Kolkata for providing me with the necessary tools and modules required for my training.

I am also grateful to my teachers and faculties back at my college, Sikkim Manipal Institute of Technology, for encouraging me to pursue a training course in Android App Development, which is something different from the general curriculum of my department.

It is an honour for me to extend my heartfelt acknowledgement towards my friends and family without whose undeterred support this project would not have seen this day.

PROJECT TITLE

**AN
AUSTRALIAN
TOURISM
APPLICATION
CREATED
USING
ANDROID
STUDIO**



**UNDER GUIDANCE
OF
Mr. Swarup Kumar Saha**

Table of Contents:

No.	Topics	Page No
1	COVER PAGE	
2	CERTIFICATE PAGE	
3	ACKNOWLEDGEMENT	
4	TITLE OF THE PROJECT	
5	CONTENT PAGE	
6	INTRODUCTION TO ANDROID STUDIO	
7	COMPONENTS OF ANDROID STUDIO	
8	INTRODUCTION TO MY APP	
9	SNAPSHOTS OF THE APP	
10	THE BACKGROUND CODE	
11	OVERVIEW AND FUTURE SCOPE	
12	BIBLIOGRAPHY	

ANDROID STUDIO

Android Studio is the official Integrated Development Environment (IDE) for Android app development, based on [IntelliJ IDEA](#). On top of IntelliJ's powerful code editor and developer tools, Android Studio offers even more features that enhance your productivity when building Android apps, such as:

- A flexible Gradle-based build system
- A fast and feature-rich emulator
- A unified environment where you can develop for all Android devices
- Instant Run to push changes to your running app without building a new APK
- Code templates and GitHub integration to help you build common app features and import sample code
- Extensive testing tools and frameworks
- Lint tools to catch performance, usability, version compatibility, and other problems
- C++ and NDK support
- Built-in support for [Google Cloud Platform](#), making it easy to integrate Google Cloud Messaging and App Engine

SETTING UP ANDROID STUDIO:

Windows

To install Android Studio on Windows, proceed as follows:

1. If you downloaded an `.exe` file (recommended), double-click to launch it.
If you downloaded a `.zip` file, unpack the ZIP, copy the **android-studio** folder into your **Program Files** folder, and then open the **android-studio > bin** folder and launch `studio64.exe` (for 64-bit machines) or `studio.exe` (for 32-bit machines).
2. Follow the setup wizard in Android Studio and install any SDK packages that it recommends.

That's it. The following video shows each step of the setup procedure when using the recommended `.exe` download.

PROJECT DOCUMENTATION

As new tools and other APIs become available, Android Studio tells you with a pop-up, or you can check for updates by clicking **Help > Check for Update**.

Mac

To install Android Studio on your Mac, proceed as follows:

1. Launch the Android Studio DMG file.
2. Drag and drop Android Studio into the Applications folder, then launch Android Studio.
3. Select whether you want to import previous Android Studio settings, then click **OK**.
4. The Android Studio Setup Wizard guides you through the rest of the setup, which includes downloading Android SDK components that are required for development.

That's it. The following video shows each step of the recommended setup procedure.

As new tools and other APIs become available, Android Studio tells you with a pop-up, or you can check for updates by clicking **Android Studio > Check for Updates**.

For further information on how to download Android Studio for other OS, refer to the android developer's site.

Project structure

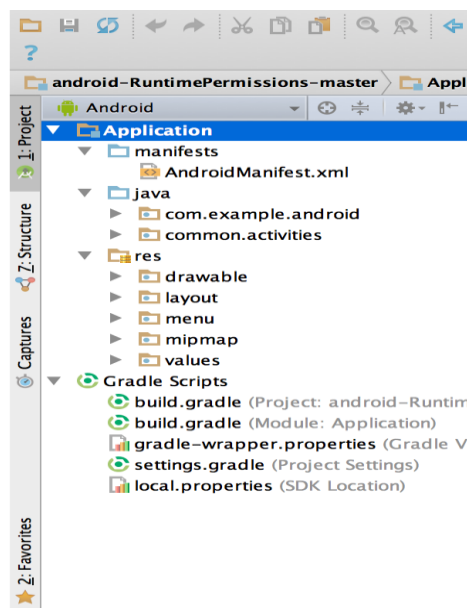


Figure: The project files in Android view.

PROJECT DOCUMENTATION

Each project in Android Studio contains one or more modules with source code files and resource files. Types of modules include:

- Android app modules
- Library modules
- Google App Engine modules

By default, Android Studio displays your project files in the Android project view, as shown in figure 1. This view is organized by modules to provide quick access to your project's key source files.

All the build files are visible at the top level under **Gradle Scripts** and each app module contains the following folders:

- **manifests:** Contains the `AndroidManifest.xml` file.
- **java:** Contains the Java source code files, including JUnit test code.
- **res:** Contains all non-code resources, such as XML layouts, UI strings, and bitmap images.

The user interface

The Android Studio main window is made up of several logical areas identified in figure shown below

PROJECT DOCUMENTATION

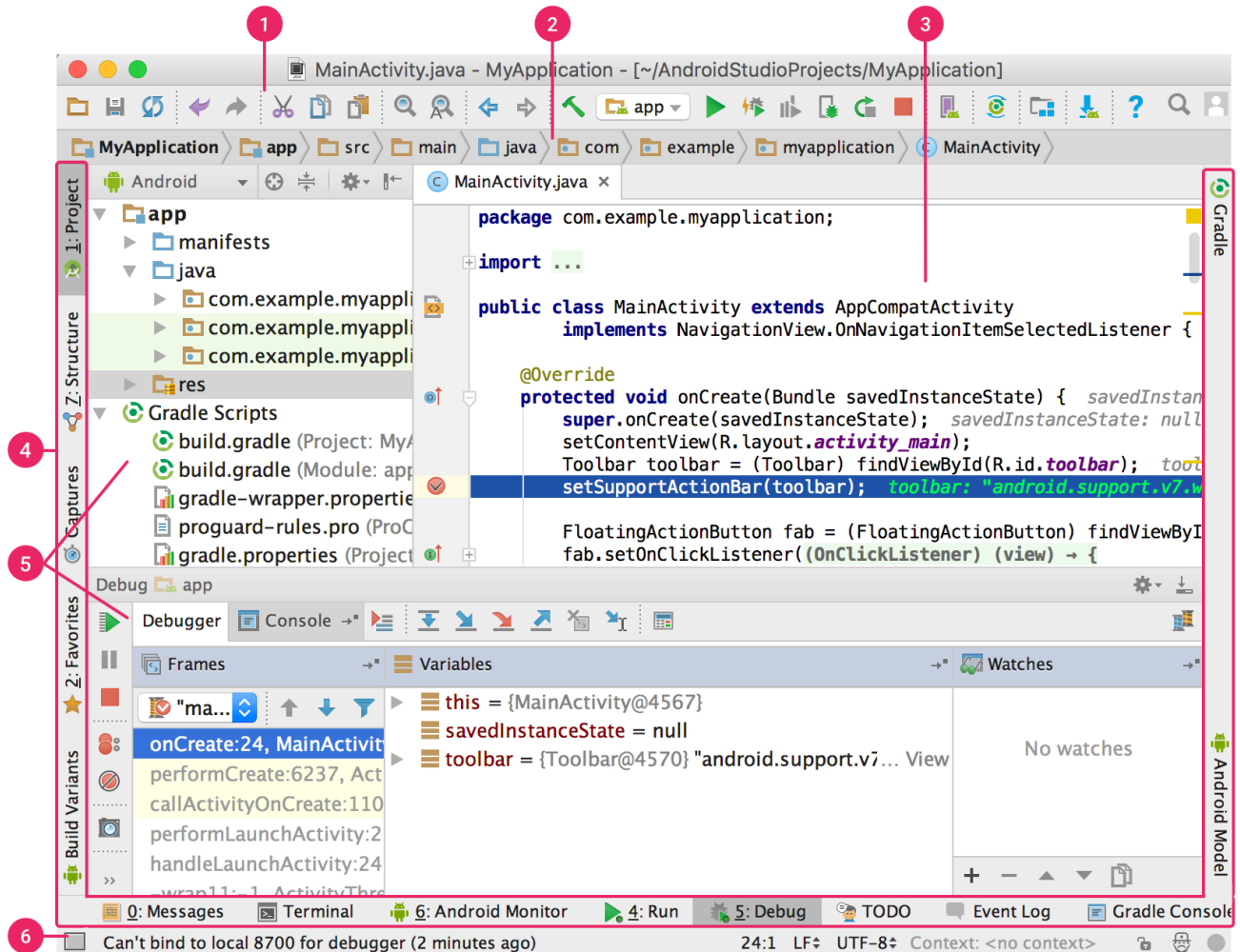


Figure: The Android Studio main window.

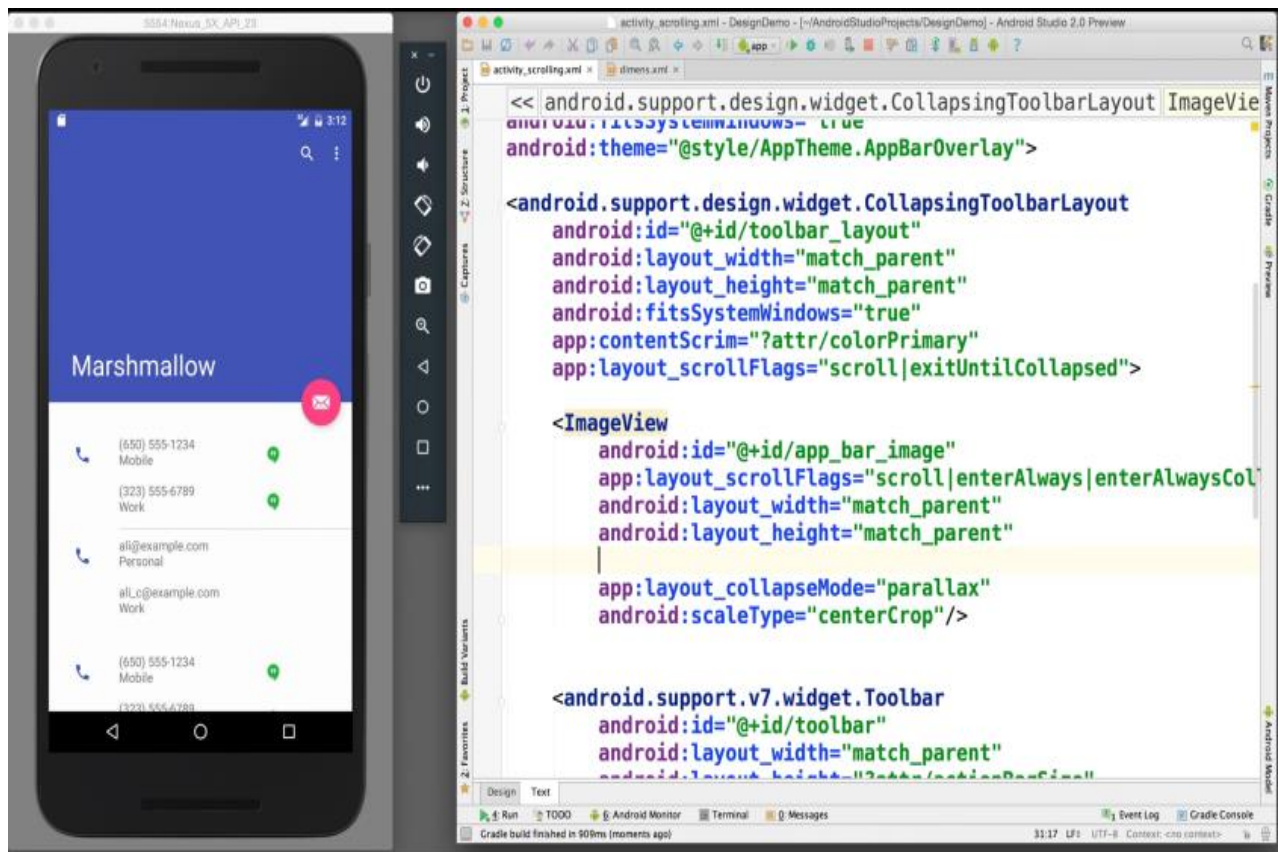
1. The **toolbar** lets you carry out a wide range of actions, including running your app and launching Android tools.
2. The **navigation bar** helps you navigate through your project and open files for editing. It provides a more compact view of the structure visible in the **Project** window.
3. The **editor window** is where you create and modify code. Depending on the current file type, the editor can change. For example, when viewing a layout file, the editor displays the Layout Editor.
4. The **tool window bar** runs around the outside of the IDE window and contains the buttons that allow you to expand or collapse individual tool windows.
5. The **tool windows** give you access to specific tasks like project management, search, version control, and more. You can expand them and collapse them.
6. The **status bar** displays the status of your project and the IDE itself, as well as any warnings or messages.

PROJECT DOCUMENTATION

You can organize the main window to give yourself more screen space by hiding or moving toolbars and tool windows. You can also use keyboard shortcuts to access most IDE features.

At any time, you can search across your source code, databases, actions, elements of the user interface, and so on, by double-pressing the Shift key, or clicking the magnifying glass in the upper right-hand corner of the Android Studio window. This can be very useful if, for example, you are trying to locate a particular IDE action that you have forgotten how to trigger.

COMPONENTS OF ANDROID STUDIO:



ACTIVITIES:

Activities are one of the fundamental building blocks of apps on the Android platform. They serve as the entry point for a user's interaction with an app, and are also central to how a user navigates within an app (as with the Back button) or between apps (as with the Recents button).

Skillfully managing activities allows you to ensure that, for example:

- Orientation changes take place smoothly without disrupting the user experience.
- User data is not lost during activity transitions.
- The system kills processes when it's appropriate to do so.

This section begins by providing an [introduction](#) to the concept of activities. It goes on to describe the [activity lifecycle](#) in detail. Next, it discusses [state changes and how to accommodate them](#). After that, this section talks about the relationship between activities and [intra-](#) and [inter-app](#) navigation. Last, this section explains the [relationship](#) between activities and the processes that host them.

INTENTS:

An [Intent](#) is a messaging object you can use to request an action from another [app component](#). Although intents facilitate communication between components in several ways, there are three fundamental use cases:

- **Starting an activity**

An [Activity](#) represents a single screen in an app. You can start a new instance of an [Activity](#) by passing an [Intent](#) to [startActivity\(\)](#). The [Intent](#) describes the activity to start and carries any necessary data.

If you want to receive a result from the activity when it finishes, call [startActivityForResult\(\)](#). Your activity receives the result as a separate [Intent](#) object in your activity's [onActivityResult\(\)](#) callback. For more information, see the [Activities](#) guide.

Intent types:

There are two types of intents:

PROJECT DOCUMENTATION

- **Explicit intents** specify which application will satisfy the intent, by supplying either the target app's package name or a fully-qualified component class name. You'll typically use an explicit intent to start a component in your own app, because you know the class name of the activity or service you want to start. For example, you might start a new activity within your app in response to a user action, or start a service to download a file in the background.
- **Implicit intents** do not name a specific component, but instead declare a general action to perform, which allows a component from another app to handle it. For example, if you want to show the user a location on a map, you can use an implicit intent to request that another capable app show a specified location on a map.

Layouts:

A layout defines the structure for a user interface in your app, such as in an [activity](#). All elements in the layout are built using a hierarchy of [View](#) and [ViewGroup](#) objects. A [View](#) usually draws something the user can see and interact with whereas a [ViewGroup](#) is an invisible container that defines the layout structure for [View](#) and other [ViewGroup](#) objects, as shown in figure.

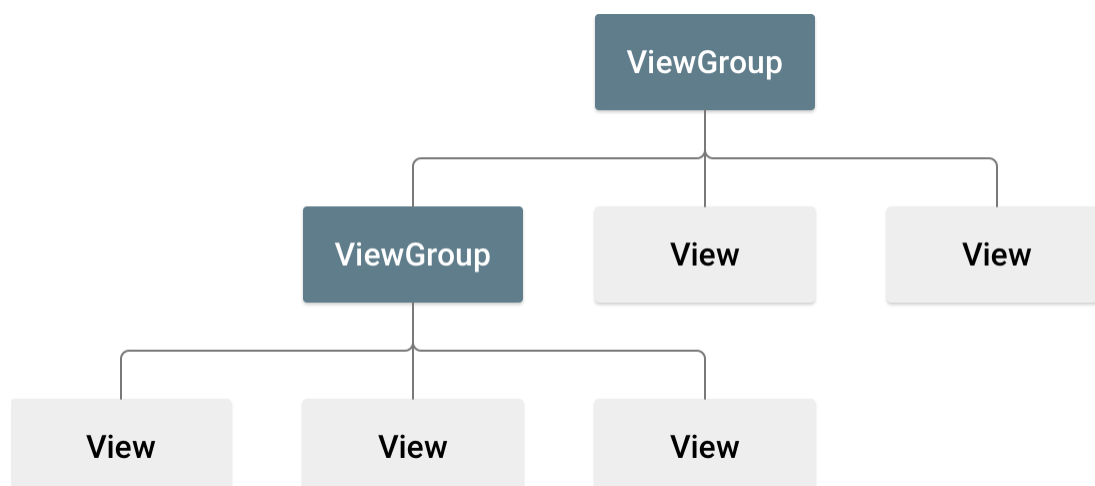


Figure: Illustration of a view hierarchy, which defines a UI layout

The [View](#) objects are usually called "widgets" and can be one of many subclasses, such as [Button](#) or [TextView](#). The [ViewGroup](#) objects are usually called "layouts" can be one of many types that provide a different layout structure, such as [LinearLayout](#) or [ConstraintLayout](#).

You can declare a layout in two ways:

- **Declare UI elements in XML.** Android provides a straightforward XML vocabulary that corresponds to the View classes and subclasses, such as those for widgets and layouts.

PROJECT DOCUMENTATION

You can also use Android Studio's [Layout Editor](#) to build your XML layout using a drag-and-drop interface.

- **Instantiate layout elements at runtime.** Your app can create View and ViewGroup objects (and manipulate their properties) programmatically.

Declaring your UI in XML allows you to separate the presentation of your app from the code that controls its behavior. Using XML files also makes it easy to provide different layouts for different screen sizes and orientations (discussed further in [Supporting Different Screen Sizes](#)).

The Android framework gives you the flexibility to use either or both of these methods to build your app's UI. For example, you can declare your app's default layouts in XML, and then modify the layout at runtime.

Styles and Themes:

Styles and themes on Android allow you to separate the details of your app design from the UI structure and behavior, similar to stylesheets in web design.

A *style* is a collection of attributes that specify the appearance for a single [View](#). A style can specify attributes such as font color, font size, background color, and much more.

A *theme* is a type of style that's applied to an entire app, activity, or view hierarchy, not just an individual view. When you apply your style as a theme, every view in the app or activity applies each style attribute that it supports.

Styles and themes are declared in a [style resource file](#) in `res/values/`, usually named `styles.xml`.

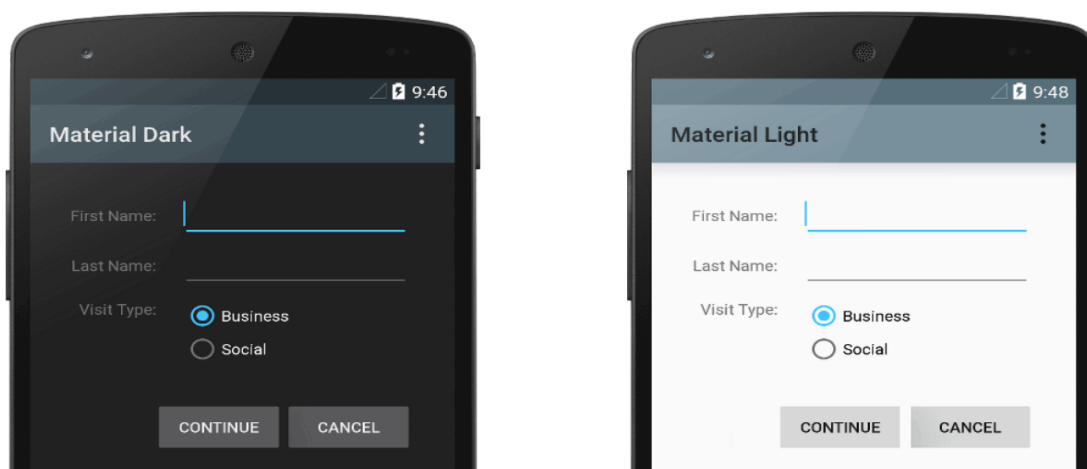


Figure: Two themes applied to the same activity: `Theme.AppCompat` (left) and `Theme.AppCompat.Light` (right)

Create and apply a style:

To create a new style or theme, open your project's `res/values/styles.xml` file. For each style you want to create, follow these steps:

1. Add a `<style>` element with a name that uniquely identifies the style.
2. Add an `<item>` element for each style attribute you want to define.

The `name` in each item specifies an attribute you would otherwise use as an XML attribute in your layout. The value in the `<item>` element is the value for that attribute.

For example, if you define the following style:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="GreenText" parent="TextAppearance.AppCompat">
        <item name="android:textColor">#00FF00</item>
    </style>
</resources>
```

You can apply the style to a view as follows:

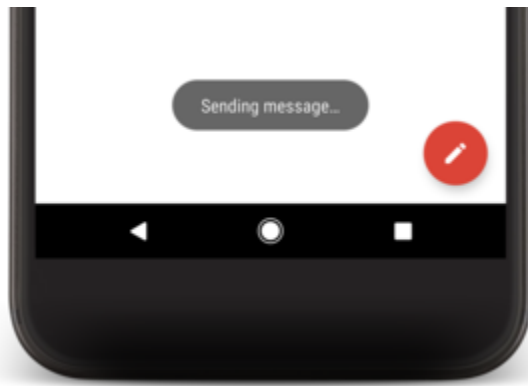
```
<TextView
    style="@style/GreenText"
    ... />
```

Each attribute specified in the style is applied to that view if the view accepts it. The view simply ignores any attributes that it does not accept.

Toasts:

A toast provides simple feedback about an operation in a small popup. It only fills the amount of space required for the message and the current activity remains visible and interactive. Toasts automatically disappear after a timeout.

For example, clicking **Send** on an email triggers a "Sending message..." toast, as shown in the following screen capture:



An example of a Toast

Alert Dialogs:

A dialog is a small window that prompts the user to make a decision or enter additional information. A dialog does not fill the screen and is normally used for modal events that require users to take an action before they can proceed.

Menus:

Menus are a common user interface component in many types of applications. To provide a familiar and consistent user experience, you should use the [Menu](#) APIs to present user actions and other options in your activities.

Beginning with Android 3.0 (API level 11), Android-powered devices are no longer required to provide a dedicated *Menu* button. With this change, Android apps should migrate away from a dependence on the traditional 6-item menu panel and instead provide an app bar to present common user actions.

Although the design and user experience for some menu items have changed, the semantics to define a set of actions and options is still based on the [Menu](#) APIs. This guide shows how to create the three fundamental types of menus or action presentations on all versions of Android:

Options menu and app bar

The [options menu](#) is the primary collection of menu items for an activity. It's where you should place actions that have a global impact on the app, such as "Search," "Compose email," and "Settings."

See the section about [Creating an Options Menu](#).

ListView:

```
public class ListView
extends AbsListView
java.lang.Object

↳ android.view.View
    ↳ android.view.ViewGroup
        ↳ android.widget.AdapterView<android.widget.ListAdapter>
            ↳ android.widget.AbsListView
                ↳ android.widget.ListView
```

Known direct subclasses

[ExpandableListView](#)

Displays a vertically-scrollable collection of views, where each view is positioned immediately below the previous view in the list. For a more modern, flexible, and performant approach to displaying lists, use [RecyclerView](#).

To display a list, you can include a list view in your layout XML file:

```
<ListView

    android:id="@+id/list_view"

    android:layout_width="match_parent"

    android:layout_height="match_parent" />
```

A list view is an [adapter view](#) that does not know the details, such as type and contents, of the views it contains. Instead list view requests views on demand from a [ListAdapter](#) as needed, such as to display new views as the user scrolls up or down.

In order to display items in the list, call `setAdapter\(ListAdapter\)` to associate an adapter with the list. For a simple example, see the discussion of filling an adapter view with text in the [Layouts](#) guide.

PROJECT DOCUMENTATION

To display a more custom view for each item in your dataset, implement a ListAdapter. For example, extend [BaseAdapter](#) and create and configure the view for each data item in `getView(...)`:

```
private class MyAdapter extends BaseAdapter {

    // override other abstract methods here

    @Override

    public View getView(int position, View convertView, ViewGroup
container) {

        if (convertView == null) {

            convertView =
getLayoutInflater().inflate(R.layout.list_item, container, false);

        }

        ((TextView) convertView.findViewById(android.R.id.text1))

            .setText(getItem(position));

        return convertView;

    }

}
```

Listview is an integral part of most apps and also forms an important constituent of the tourism app developed as a part of this project.

Drawer layout:

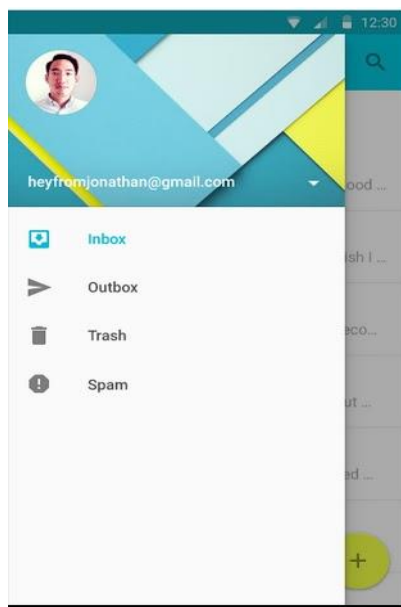
`DrawerLayout` acts as a top-level container for window content that allows for interactive "drawer" views to be pulled out from one or both vertical edges of the window.

Drawer positioning and layout is controlled using the `android:layout_gravity` attribute on child views corresponding to which side of the view you want the drawer to emerge from: left or right (or start/end on platform versions that support layout direction.) Note that you can only have one drawer view for each vertical edge of the window. If your layout configures more than one drawer view per vertical edge of the window, an exception will be thrown at runtime.

To use a `DrawerLayout`, position your primary content view as the first child with width and height of `match_parent` and no `layout_gravity`>. Add drawers as child views after the main content view and set the `layout_gravity` appropriately. Drawers commonly use `match_parent` for height with a fixed width.

[`DrawerLayout.DrawerListener`](#) can be used to monitor the state and motion of drawer views. Avoid performing expensive operations such as layout during animation as it can cause stuttering; try to perform expensive operations during the `STATE_IDLE` state. [`DrawerLayout.SimpleDrawerListener`](#) offers default/no-op implementations of each callback method.

As per the [Android Design guide](#), any drawers positioned to the left/start should always contain content for navigating around the application, whereas any drawers positioned to the right/end should always contain actions to take on the current content. This preserves the same navigation left, actions right structure present in the Action Bar and elsewhere.



MY APP

AUSTRALIA TOURISM:

Tourism in Australia is an important component of the [Australian economy](#). In the financial year 2014/15, [tourism](#) represented 3.0% of [Australia's GDP](#) contributing A\$47.5 billion to the national economy.^[2] Domestic tourism is a significant part of the tourism industry, representing 73% of the total direct tourism GDP.^[2]

In calendar year 2015, there were 7.4 million visitor arrivals.^[3] Tourism employed 580,800 people in Australia in 2014-15, 5% of the workforce.^[2] About 43.7% of persons employed in tourism were part-time. Tourism also contributed 8.0% of Australia's total export earnings in 2010-11.^[2]

Popular Australian destinations include the coastal cities of [Sydney](#), [Brisbane](#) and [Melbourne](#), as well as other high-profile destinations including regional [Queensland](#), the [Gold Coast](#) and the [Great Barrier Reef](#), the world's largest reef. [Uluru](#) and the [Australian outback](#) are other popular locations, as is the [Tasmanian wilderness](#). The unique Australian wildlife is also another significant point of interest in the country's tourism.

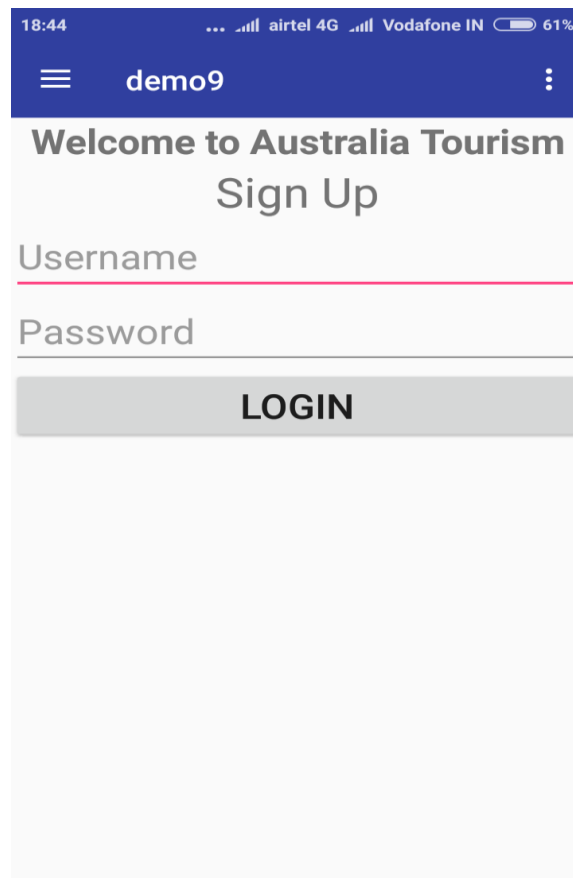
Despite the global economic challenges and natural disasters in 2010-2011, Australia's tourism growth was supported by increased consumption (up 4.4% over the previous year, largely due to an increase in the number of visitors from overseas). On the back of a strong Australian dollar, 2010-11 also saw a record 7.4 million short-term resident departures from Australia, an increase of 9.9% from 2009-10. Domestic tourism consumption grew at less than half the pace of international consumption in 2010-11 (up 2.1% compared to 4.4%).^[4]

The Australian Government released the 2020 Tourism Industry Potential on 15 November 2010, and estimates the Australian tourism industry to be worth up to \$140 billion in overnight expenditure. This growth will largely be due to key emerging markets, including the China market, which is estimated to be the largest economic contributor to the Australian tourism industry by 2020.^[5] The number of Chinese visitors has more than doubled from 2006 to 2012 reaching a peak of 626,400 in 2012 and surpassing for the first time the number of arrivals from the United Kingdom.^[6] In 2013, China was Australia's fastest growing tourist market.^[7] According to Tourism Australia Managing Director Andrew McEvoy, the Chinese are the highest spending visitors to the country.^[8]

SNAPSHOTS OF THE APPLICATION:

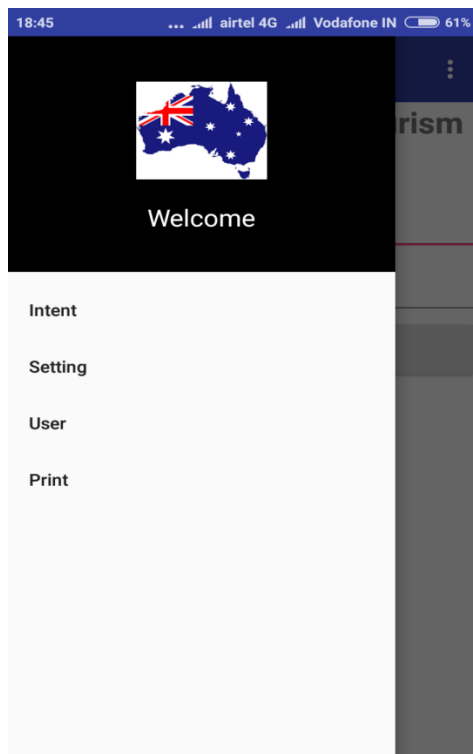
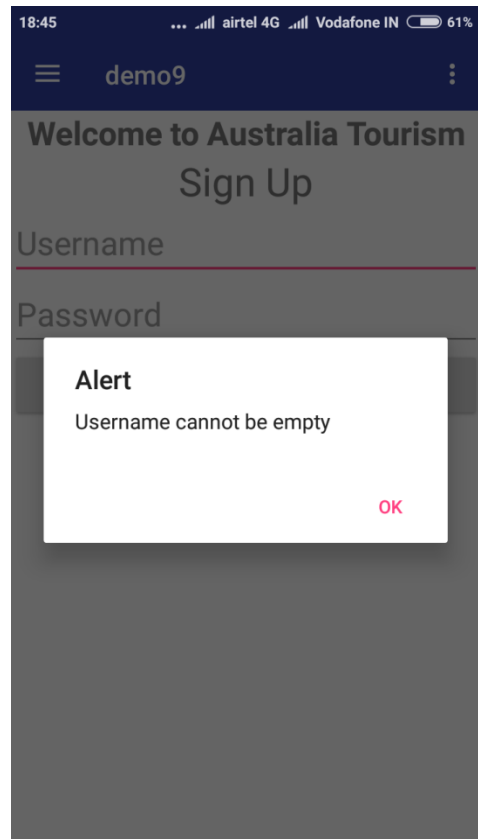


The Splash Screen which loads when the application is started.



The Main Activity

The Alert Dialog Box
which pops up when
username is left blank



The Drawer Layout showing
the Navigation View

A ListView showing the different districts of the Australian Territory alongwith their respective Capitals.

18:46

... airtel 4G Vodafone IN 61%



Queensland

Queensland



New South Wales

Sydney



Victoria

Melbourne



Western Australia

Perth



South Australia

Adelaide

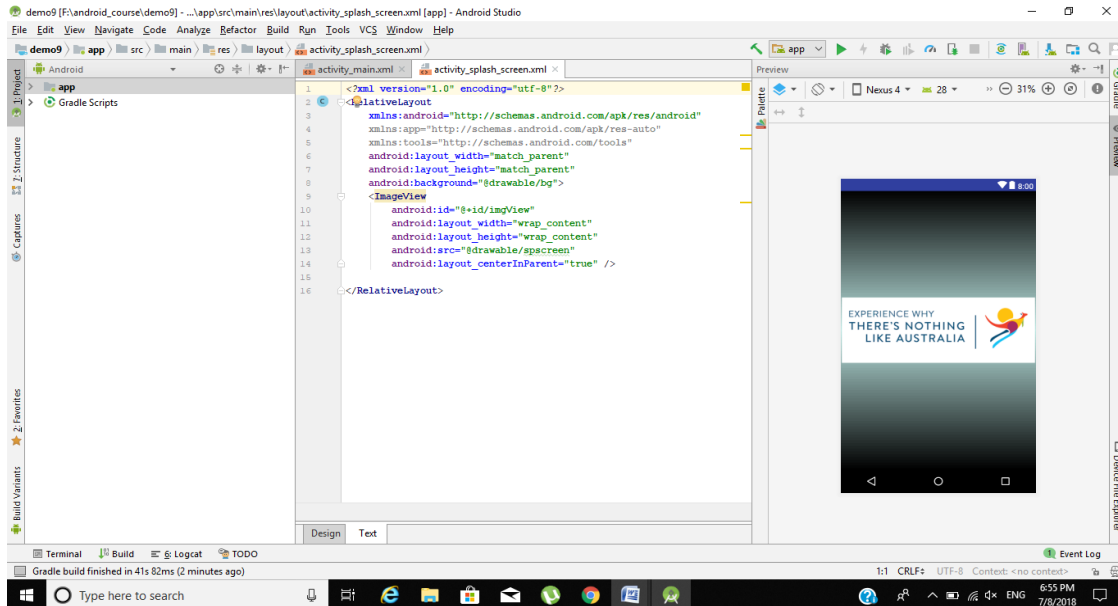


Northern Territory

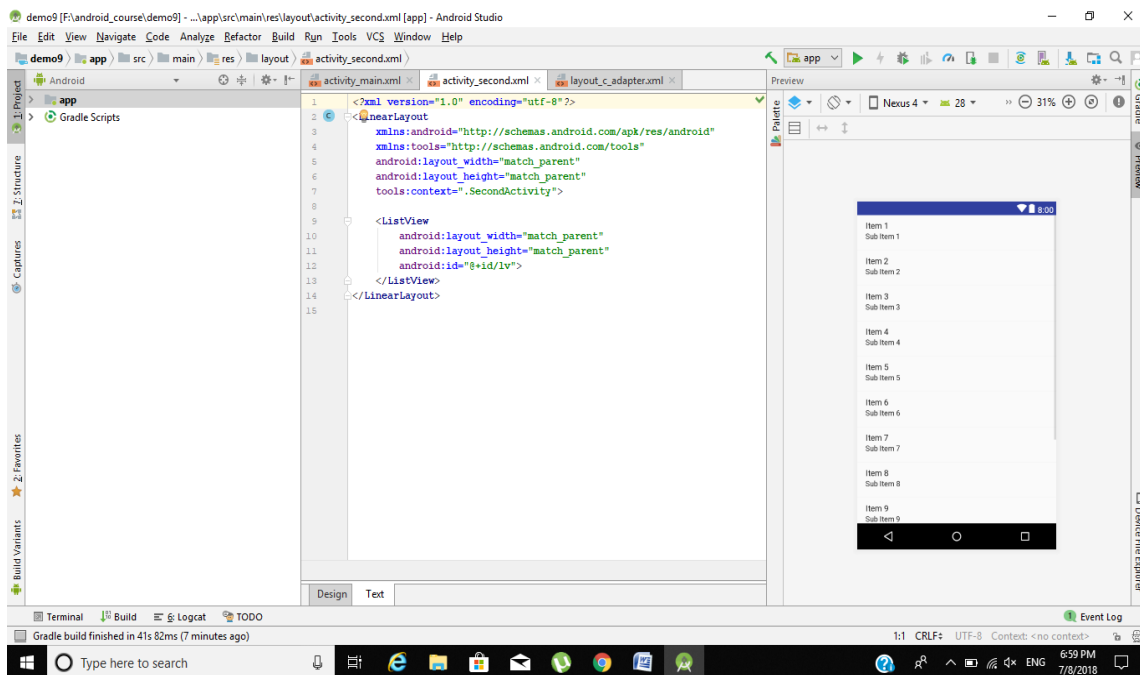
Darwin

THE BACKGROUND CODING:

- The Splash Screen Activity's .xml file:

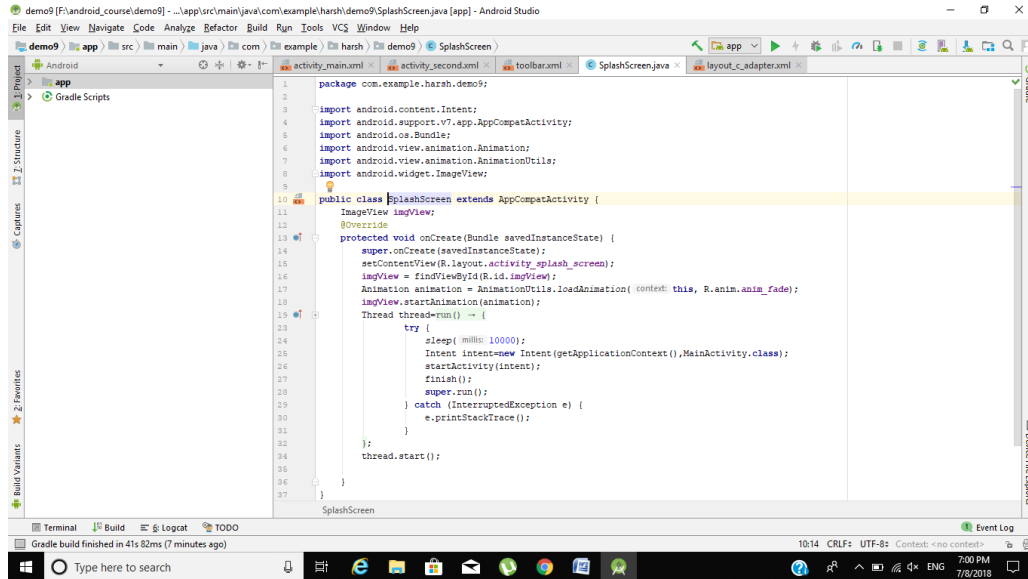


- The ListView Activity's .xml file:

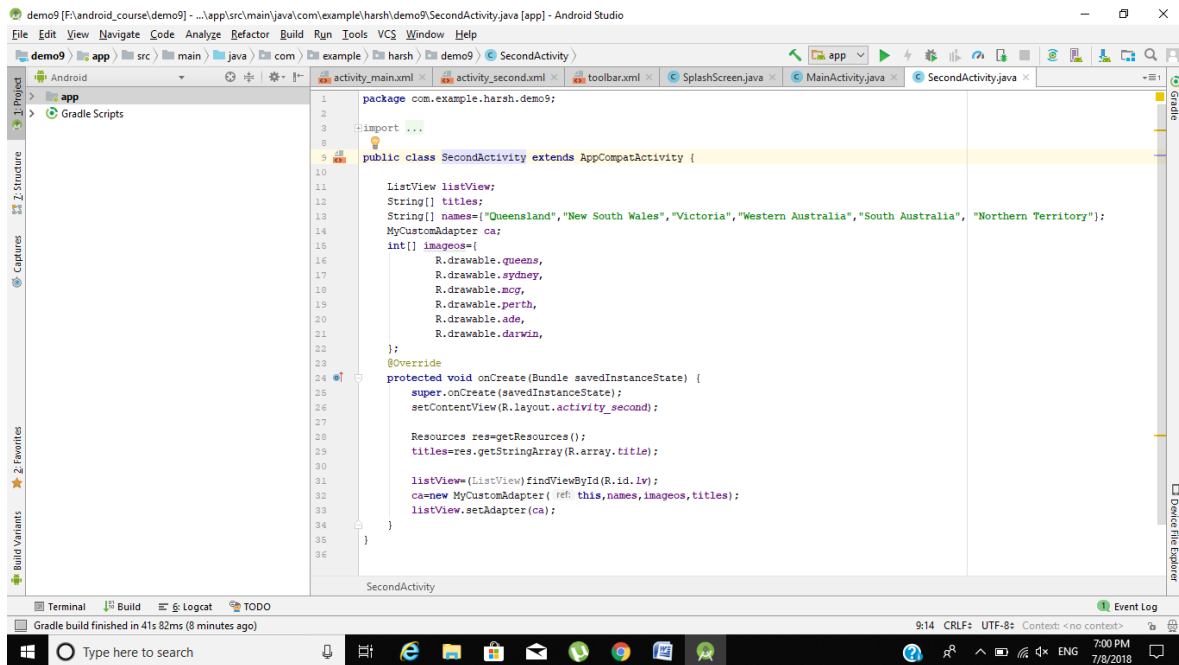


PROJECT DOCUMENTATION

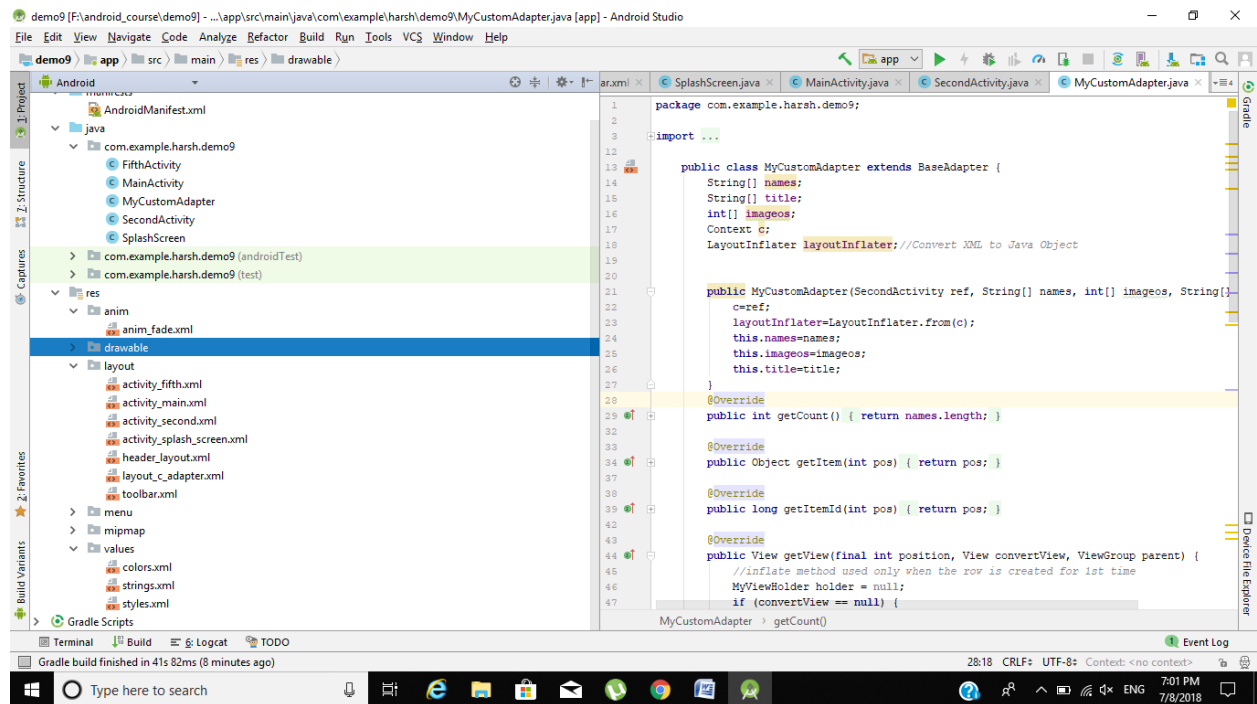
- The Splash Screen activity's .java file



- The ListView Activity's .java file:



A screenshot showing all the .xml files and the .java files used in the project for the development of the concerned app.



Overview:

The application provides a comprehensive synopsis of the topics that were covered in the training undertaken and involves the usage of splash screens, activities, styles and strings, manifest file, intents, listview, alert dialog boxes, widgets, viewgroups, toasts, menus, drawer layout, navigation view and some other features.

A user logging into the application is greeted by a splash screen with a duration of 10 seconds and is inspired by the official logo of Tourism Australia. The main activity has a toolbar featuring a menu icon, hamburger icon and the app name. The user can log into the application or can go to the hamburger menu and select the intent option to be led into the listview showed above. Each item in the listview is further customizable and can contain information regarding the selected item in the listview. This is a very simple application aimed at showing the minimal knowledge the students have gained in this extremely short time and has scope for further improvement.

Usage:

This application is very simple and provides the user with basic information about the places of interest in the Australian Sub-continent.

Future Scope and Further Enhancement of the Project:

The project has a huge future prospect and if worked and developed upon can become one which can be confidently uploaded on Play Store and may find a future if the general public accepts it with warmth for gaining information on Australian Tourism. However, that road for this application is long and can take quite some time and dedication and can only be achieved by gaining more and more knowledge and trying out more and more things. In the end, I just hope that I keep going from strength and strength and reach a stage where I can call myself an Android developer.

Bibliography –

Given below are the names of some books, which I've used:

- **Barry Burd**- "Android Application Development"- from Wiley.
- **Pradeep Kothari** – "Android Application Development"- from dreamtech Publications

The websites which I have referred to are:

- www.developer.android.com
- www.google.com
- www.wikipedia.com